

Angular Frontend Architecture Guide

Generated on 2026-04-20 12:38:36 from the current workspace. This guide explains the purpose, structure, internal working, service connections, and important files for **connectsphere-web**.

The Angular frontend is a lightweight assessment-ready client designed to stay readable while still touching all major platform services. It has a social-dashboard layout inspired by Twitter, Instagram, and Facebook, but keeps the code intentionally simple.

The app contains feed, explore, profile, notifications, stories, and chat pages, plus a shared session panel that handles login and registration through auth-service.

Architecture Snapshot

Module	connectsphere-web
Type	Angular standalone frontend
Port / Entry	4200 (served through gateway paths on 8080 during development)
Data Ownership	None
Protocols / Main Tech	Angular, HttpClient, router, STOMP/SockJS
Verification Command	npm run build and npm test -- --watch=false
Key Endpoints or Routes	/feed /explore /profile /notifications /stories /chat /oauth2/authorization/google /oauth2/authorization/github

Responsibilities

- Host a clean UI shell and route pages
- Store auth session locally and send bearer tokens
- Call backend services through the gateway

- Render stories, notifications, follow suggestions, and realtime chat
- Provide a simple but attractive microservice demo surface

Integrations

- Uses gateway-relative URLs everywhere
- Uses auth-service for session state
- Uses post/comment/like/follow/notification/media/search/chat services across pages
- Uses proxy config for local Angular development

Internal Working

- Session flow: the right-side shell logs users in or registers them through auth-service, stores the JWT pair in local storage, and refreshes the profile into reactive session state.
- Feed flow: the feed page loads followed-user relationships, requests post-service feed data, then orchestrates comment-service and like-service actions from one UI card.
- Explore flow: the page combines auth-service user search, post-service delegated search through search-service, trending hashtags, and follow suggestions.
- Stories flow: the stories page uploads multipart files to media-service and previews active stories returned from the service.
- Chat flow: the chat page uses REST endpoints for history and STOMP/SockJS for live conversation updates through the gateway websocket route.

File Inventory

The table below lists the important source, configuration, and test files that currently exist in the module. This is meant to help you explain how the module is connected internally when you present it or revise it later.

File	Description and why it matters
angular.json	Angular workspace configuration describing build targets, dev-server behavior, proxying, and optimization settings.

File	Description and why it matters
package.json	Frontend dependency and script manifest used to install, build, serve, and test the Angular application.
README.md	Human-readable runbook describing what the module does, how to start it, and what endpoints or features it exposes.
src\app\app.config.ts	Supporting source file that contributes to the module's implementation, build, or verification story.
src\app\app.html	Root Angular shell for global layout, session controls, and router hosting.
src\app\app.routes.ts	Angular route table mapping URL paths to the major frontend pages.
src\app\app.scss	Root Angular shell for global layout, session controls, and router hosting.
src\app\app.spec.ts	Supporting source file that contributes to the module's implementation, build, or verification story.
src\app\app.ts	Root Angular shell for global layout, session controls, and router hosting.
src\app\core\chat-realtime.service.ts	Shared Angular client logic used across multiple pages for session state, API requests, models, and realtime communication.
src\app\core\connectsphere-api.service.ts	Shared Angular client logic used across multiple pages for session state, API requests, models, and realtime communication.
src\app\core\session.service.ts	Shared Angular client logic used across multiple pages for session state, API requests, models, and realtime communication.
src\app\core\social.models.ts	Shared Angular client logic used across multiple pages for session state, API requests, models, and realtime communication.
src\app\pages\chat\chat.html	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\chat\chat.scss	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\chat\chat.ts	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\explore\explore.html	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\explore\explore.scss	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.

File	Description and why it matters
src\app\pages\explore\explore.ts	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\feed\feed.html	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\feed\feed.scss	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\feed\feed.ts	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\notifications\notifications.html	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\notifications\notifications.scss	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\notifications\notifications.ts	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\profile\profile.html	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\profile\profile.css	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\profile\profile.ts	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\stories\stories.html	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\stories\stories.css	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\app\pages\stories\stories.ts	Standalone Angular route/page implementing one user-facing slice of the ConnectSphere frontend.
src\index.html	Supporting source file that contributes to the module's implementation, build, or verification story.
src\main.ts	Supporting source file that contributes to the module's implementation, build, or verification story.

File	Description and why it matters
src\proxy.conf.json	Angular dev-server proxy so frontend requests can reach the gateway and websocket endpoints during local development.
src\styles.scss	Supporting source file that contributes to the module's implementation, build, or verification story.

Assessment Notes

Current file inventory count: **35**

This guide focuses on how the module works end to end: what it owns, which requests it handles, what supporting files make it run, and how it connects to the rest of ConnectSphere.